Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Criação de um motor gráfico

Luís Ferreira a98286
José Fernandes a106937
Gabriel Dantas a107291
Simão Oliveira a107322

Fevereiro, 2026

Índice

1. Introdução **1**

2. Generator **2**

2.1. Base do generator 2
2.2. Plano 3
2.3. Caixa 4
2.4. Esfera 5
2.5. Cone 6
2.6. Cilindro 8
2.7. Esfera gerada através de um icosaedro 9
2.8. Toro 11
2.9. Formato dos ficheiros dos modelos 12

3. Engine **14**

3.1. Parsing dos dados 14
3.2. Geração das Figuras 15
3.3. Interação com a Cena 15

4. Resultados obtidos **16**

4.1. Figuras Geométricas Geradas 16
4.2. Testes Fornecidos 17

5. Conclusão **18**

Bibliografia **19**

Lista de Siglas e Acrónimos **20**

Anexos **21**

Anexo 1 Logo da Universidade do Minho 21

Lista de Figuras

Figura 1 Representação de um *quad*. 2
Figura 2 Representação do plano. 3
Figura 3 Representação do cubo. 4
Figura 4 Representação da esfera. 5
Figura 5 Representação da base do cone. 6
Figura 6 Representação do cone. 7
Figura 7 Representação do cilindro. 8
Figura 8 Representação de um icosaedro. 9
Figura 9 Cálculo dos pontos médios de uma das faces. 10
Figura 10 Pontos médios após normalização. 11
Figura 11 Aproximação à esfera utilizando o método descrito. 11
Figura 12 Aproximação desejada de construir do toro. 12
Figura 13 Esquema de representação das variáveis. 12
Figura 14 Teste plano. 16
Figura 15 Teste box. 16
Figura 16 Teste cone. 16
Figura 17 Teste cilindro. 16
Figura 18 Teste esfera. 16
Figura 19 Teste esfera (gerada através de um icosaedro). 16
Figura 20 Teste torus. 17
Figura 21 Teste cena 1. 17
Figura 22 Teste cena 2. 17
Figura 23 Teste cena 3. 17
Figura 24 Teste cena 4. 17
Figura 25 Teste cena 5. 17

1. Introdução

O presente relatório pretende descrever o processo de desenvolvimento do trabalho laboratorial desenvolvido no âmbito da Unidade Curricular de Computação Gráfica.

Com base no enunciado fornecido o objetivo é criar um sistema completo de geração e visualização de cenas tridimensionais, composto por duas aplicações principais: o *generator*, responsável pela criação de modelos geométricos, e a *engine*, encarregue da leitura de ficheiros XML, carregamento de modelos e renderização das cenas utilizando a API *OpenGL* [1]. O trabalho é desenvolvido de forma incremental, estando dividido em quatro fases distintas, cada uma introduzindo novos conceitos fundamentais da Computação Gráfica.

Na 1ª fase, é necessário implementar a leitura do ficheiro XML no arranque da aplicação e criar estruturas de dados adequadas para armazenar a informação da cena em memória. O *generator* deve produzir modelos geométricos básicos e a *engine* deve conseguir carregá-los e desenhá-los.

2. Generator

Comece-se por descrever o funcionamento do programa *generator*.

2.1. Base do generator

A ideia básica do *generator* é ser um programa independente da *engine*, conseguindo criar e armazenar conjuntos de pontos que representam triângulo pertencentes à formação de certas figuras geométricas.

Durante a primeira fase o objetivo é gerar quatro primitivas gráficas simples: planos, caixas, esferas e cones. Para criar os ficheiros com os modelos destas figuras o *generator* vai sempre receber como parâmetros a primitiva desejada, alguns parâmetros relativos à dimensão e nível de detalhe desta e o arquivo para onde o *output* deve ser escrito. Este arquivo, que representa os modelos geométricos, é definido como sendo uma extensão *.3d*.

O programa foi criado em C++ e tem um funcionamento bastante simples. Existe um *parsing* e verificação de informações passadas como argumento. Se não existir qualquer inconformidade nestas, é gerada a figura desejada consoante a primitiva pedida.

Todas as funções de geração usam uma função auxiliar que gera um *quad* (polígono de quatro lados e quatro vértices), pelo que achamos pertinente explicar o funcionamento desta função. Considere-se qualquer ponto, p_n, tal que $n \in \mathbb{N}^+$, onde n apenas representa uma numeração arbitrária para os pontos. O ponto p_n tem coordenadas (x_n, y_n, z_n).

Dado que as *engines* gráficas interpretam uma das faces como a face correta das figuras, temos que ter atenção à ordem com que desenhamos os pontos para formar triângulos. Para que a face que será desenhada esteja visível para nós, precisamos de desenhar os pontos em sentido contra relógio.

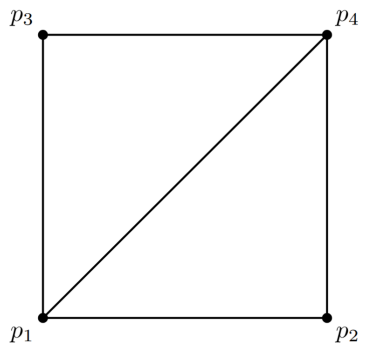


Figura 1: Representação de um *quad*.

Suponha-se o *quad* da Figura 1. Um *quad* é facilmente desenhado através de dois triângulos, propriedade útil, visto que a nossa *engine* será desenvolvida de modo a apenas interpretar triângulos. Para que a figura seja visível do lado pretendido, teremos que desenhar os dois triângulos que o constituem na ordem p_1, p_4, p_3 e p_1, p_2, p_4.

Dado isto, passemos a explicar a geração dos pontos para cada primitiva.

2.2. Plano

Para o plano, a geração dos pontos é bastante simples. Usa os parâmetros de tamanho, seja t e o número de divisões por aresta, seja d. Dado que o plano está no plano X Z, podemos assumir, para todos os pontos, que $y_n = 0$. Pensemos, então, apenas no subconjunto das coordenadas (x, z).

Como o plano estará centrado em $(0, 0)$, sabemos que os pontos limite do plano serão os quatro pontos $(\pm\frac{t}{2}, \pm\frac{t}{2})$.

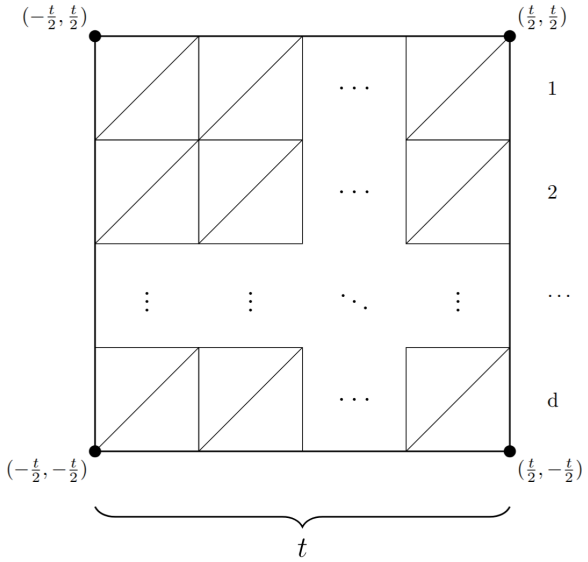


Figura 2: Representação do plano.

Dado isto, basta apenas dividir o comprimento (t) pelo número de divisões por aresta (d), obtendo a distância entre os vértices dos quadrados da malha da figura. Iterando pelo número de divisões e considerando sempre os pontos $(x_0, z_0) = \left(-\frac{t}{2} + \frac{t}{d} * i, -\frac{t}{2} + \frac{t}{d} * j\right)$ e $(x_1, z_1) = \left(-\frac{t}{2} + \frac{t}{d} * (i+1), -\frac{t}{2} + \frac{t}{d} * (j+1)\right)$, com i sendo um iterador externo e j um iterador interno, conseguiremos, em cada iteração, obter os quatro vértices de um dos quadrados da malha.

Tendo esta informação basta, a cada iteração, desenhar os quadrados que representam a face superior e a face inferior do plano, ou seja, respetivamente, os quadrados em que p_1, p_2, p_3 e p_4, da Figura 1 correspondem a $(x_0, z_0), (x_1, z_0), (x_0, z_1), (x_1, z_1)$ e $(x_1, z_0), (x_0, z_0), (x_1, z_1), (x_0, z_1)$.

A Figura 2 esquematiza esta construção.

2.3. Caixa

A caixa, à semelhança do plano, recebe, passado como argumento, o tamanho de uma aresta, seja, por semelhança ao exemplo acima, t, e o número de divisões de cada aresta, seja d.

Da mesma maneira que o plano, vamos desenhar o cubo centrado na origem. Logo, sabemos que os oito vértices do cubo serão $(\pm\frac{t}{2}, \pm\frac{t}{2}, \pm\frac{t}{2})$.

Uma vez que esta figura é fechada, não precisamos de desenhar as faces internas dela, pelo que apenas nos preocuparemos com as seis faces externas.

Ora, à semelhança do plano, basta ter dois iteradores que vão controlando o desenho dos quadrados que formam a malha em cada face do cubo, usando a função auxiliar falada que gera os *quads*. O resultado é a geração simultânea das várias faces da caixa. Veja-se a Figura 3 que demonstra bem esta lógica, mantendo uma numeração da ordem de geração. Sejam $a_0 = -\frac{t}{2} + \frac{t}{d} * i, b_0 = -\frac{t}{2} + \frac{t}{d} * j, a_1 = -\frac{t}{2} + \frac{t}{d} * (i + 1), b_1 = -\frac{t}{2} + \frac{t}{d} * (j + 1)$.

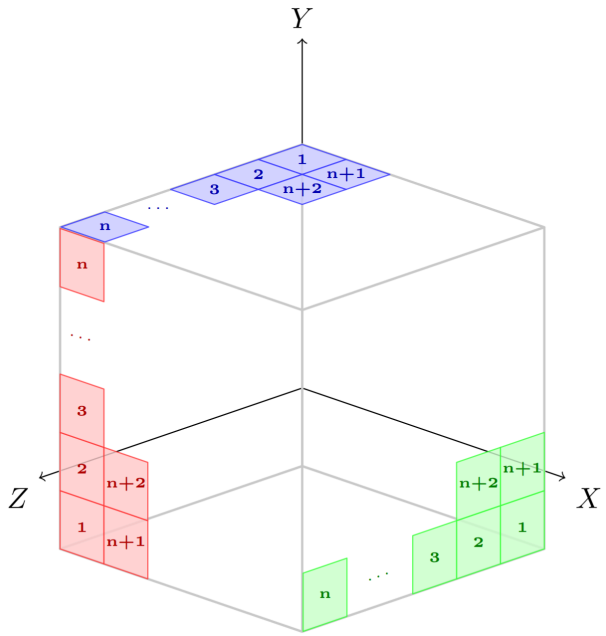


Figura 3: Representação do cubo.

Assim, geraremos sempre os quadrados:

- $(a_0, b_0, \frac{t}{2}), (a_1, b_0, \frac{t}{2}), (a_0, b_1, \frac{t}{2}), (a_1, b_1, \frac{t}{2})$, que representa a face frontal;
- $(a_0, b_0, -\frac{t}{2}), (a_1, b_0, -\frac{t}{2}), (a_0, b_1, -\frac{t}{2}), (a_1, b_1, -\frac{t}{2})$, que representa a face traseira;
- $(\frac{t}{2}, b_0, a_0), (\frac{t}{2}, b_0, a_1), (\frac{t}{2}, b_1, a_0), (\frac{t}{2}, b_1, a_1)$, que representa a face direita;
- $(-\frac{t}{2}, b_0, a_0), (-\frac{t}{2}, b_0, a_1), (-\frac{t}{2}, b_1, a_0), (-\frac{t}{2}, b_1, a_1)$, que representa a face esquerda;
- $(a_0, \frac{t}{2}, b_1), (a_1, \frac{t}{2}, b_1), (a_0, \frac{t}{2}, b_0), (a_1, \frac{t}{2}, b_0)$, que representa a face superior;
- $(a_0, -\frac{t}{2}, b_0), (a_1, -\frac{t}{2}, b_0), (a_0, -\frac{t}{2}, b_1), (a_1, -\frac{t}{2}, b_1)$, que representa a face inferior.

2.4. Esfera

A esfera recebe como parâmetro o seu raio, seja r, o número de fatias (*slices*) em que se irá particionar, seja s e o número de pilhas (*stacks*) em que se irá particionar, seja p.

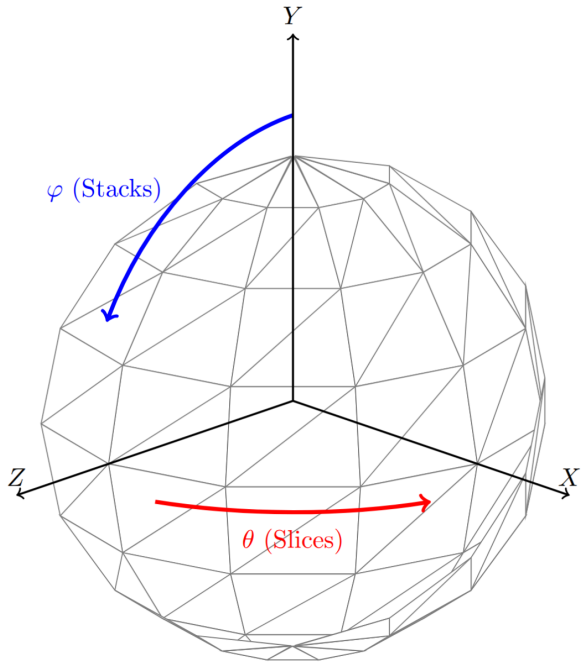


Figura 4: Representação da esfera.

Na Figura 4 representa-se a aproximação à esfera que se deseja fazer. O θ representa o eixo segundo o qual ocorrem as variações de valores fixos de *slices* (em que $0^{\underline{o}} \leq \theta \leq 360^{\underline{o}}$) e o φ o eixo segundo o qual ocorrem as variações de valores fixos de *stacks* (em que $0^{\underline{o}} \leq \varphi \leq 180^{\underline{o}}$).

Usar-se-á, com vista à representação desta, um sistema de coordenadas esféricas definido por:

$$\begin{cases} x = r * \sin(\varphi) * \cos(\theta) \\ y = r * \cos(\varphi) \\ z = r * \sin(\varphi) * \sin(\theta) \end{cases}$$

A ideia é pré-computar os espaçamentos entre ângulos horizontal e verticalmente e utilizar um iterador interno e um externo para criar os *quads* [2]. Estes espaçamentos angulares, definem-se como:

$$\begin{cases} \Delta\theta = \frac{2\pi}{s} \\ \Delta\varphi = \frac{\pi}{p} \end{cases}$$

Iterando sobre i e j, teremos os argumentos auxiliares $\varphi_1 = i * \Delta\varphi$, $\varphi_2 = (i+1) * \Delta\varphi$, $\theta_1 = j * \Delta\theta$ e $\theta_2 = (j+1) * \Delta\theta$.

Com base nestes ângulos, basta desenhar os *quads* do quadrado com os pontos (designados tal como na Figura 1):

$$\begin{cases} p_1 = r * (\sin(\varphi_1) * \cos(\theta_1), \cos(\varphi_1), \sin(\varphi_1) * \sin(\theta_1)) \\ p_2 = r * (\sin(\varphi_1) * \cos(\theta_2), \cos(\varphi_1), \sin(\varphi_1) * \sin(\theta_2)) \\ p_3 = r * (\sin(\varphi_2) * \cos(\theta_1), \cos(\varphi_2), \sin(\varphi_2) * \sin(\theta_1)) \\ p_4 = r * (\sin(\varphi_2) * \cos(\theta_2), \cos(\varphi_2), \sin(\varphi_2) * \sin(\theta_2)) \end{cases}$$

Por uma questão de otimização decidimos, nos casos em que $i = 0$ e $i = s - 1$, ou seja, os casos dos *quads* que fechavam a esfera no eixo Y, desenhar apenas triângulos. Isto é preferível porque nestes casos os *quads* convergem para uma figura triangular, não havendo necessidade de desenhar dois triângulos.

Para o caso de $i = 0$

$$\begin{cases} p_1 = (0, r, 0) \\ p_2 = r * (\sin(\varphi_2) * \cos(\theta_2), \cos(\varphi_2), \sin(\varphi_2) * \sin(\theta_2)) \\ p_3 = r * (\sin(\varphi_2) * \cos(\theta_1), \cos(\varphi_2), \sin(\varphi_2) * \sin(\theta_1)) \end{cases}$$

Para o caso de $i = s - 1$:

$$\begin{cases} p_1 = r * (\sin(\varphi_1) * \cos(\theta_1), \cos(\varphi_1), \sin(\varphi_1) * \sin(\theta_1)) \\ p_2 = r * (\sin(\varphi_1) * \cos(\theta_2), \cos(\varphi_1), \sin(\varphi_1) * \sin(\theta_2)) \\ p_3 = (0, -r, 0) \end{cases}$$

De notar que estas coordenadas respeitam a ordem das faces que desejamos para que sejam visíveis corretamente.

2.5. Cone

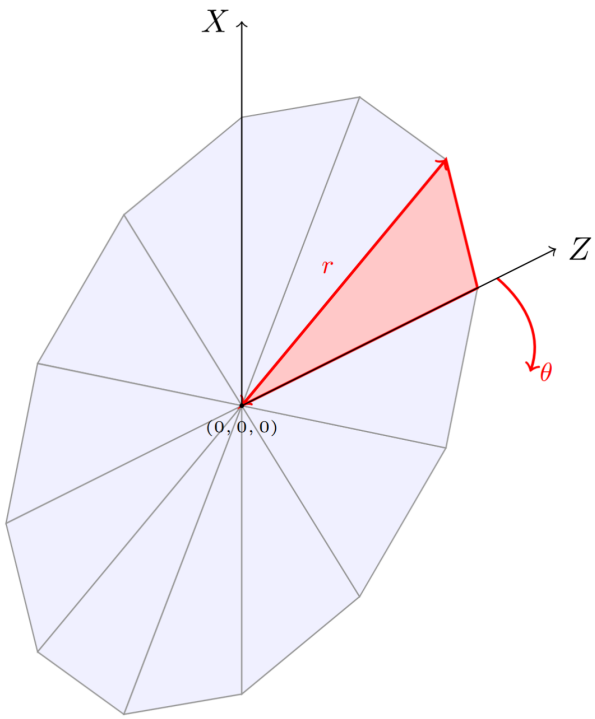


Figura 5: Representação da base do cone.

Para a criação do cone recebemos como parâmetros o raio da base do cone, seja r, a altura do cone, seja h, o número de fatias (*slices*) em que se irá particionar, seja s e o número de pilhas (*stacks*) em que se irá particionar, seja p.

Como se observa na Figura 5, para criar a base do cone, temos auxílio do ângulo θ, que representa o eixo segundo qual ocorre a criação das *slices* (em que $0^{\underline{o}} \leq \theta \leq 360^{\underline{o}}$). Iteramos pelo número de *slices*, usando $\Delta\theta = \frac{2\pi}{s}$ e fazendo $\theta_1 = i * \Delta\theta$ e $\theta_2 = (i + 1) * \Delta\theta$.

A cada passo da iteração, desenhamos o triângulo com os pontos $(r * \cos(\theta_1), 0, r * \sin(\theta_1))$, $(r * \cos(\theta_2), 0, r * \sin(\theta_2))$ e $(0, 0, 0)$.

Após isto, teremos um iterador externo, i, e um iterador interno, j, para desenhar o cone em altura. Usa-se uma interpolação linear para calcular o raio nas diferentes alturas delimitadas pelas *stacks* (representada na Figura 6 por l) [3].

$$r_k = r * \left(1 - \frac{k}{p}\right)$$

Sendo $k \in [0, p - 1]$, representando a fixação do valor de y. Assim, r_k calcula o tamanho do raio da circunferência no novo valor y. Com este método, calculamos, no ciclo externo, os raios de duas *stacks* sequenciais $r_i = r * \left(1 - \frac{i}{p}\right)$ e $r_{i+1} = r * \left(1 - \frac{i+1}{p}\right)$.

No ciclo interno calculamos $\Delta\theta$ (tal como para desenhar a base do cone), o que nos permite calcular os respetivos pontos para desenhar um *quad* (dois triângulos) da malha. Vejam-se os pontos (ordenados como na Figura 1) abaixo que o representam.

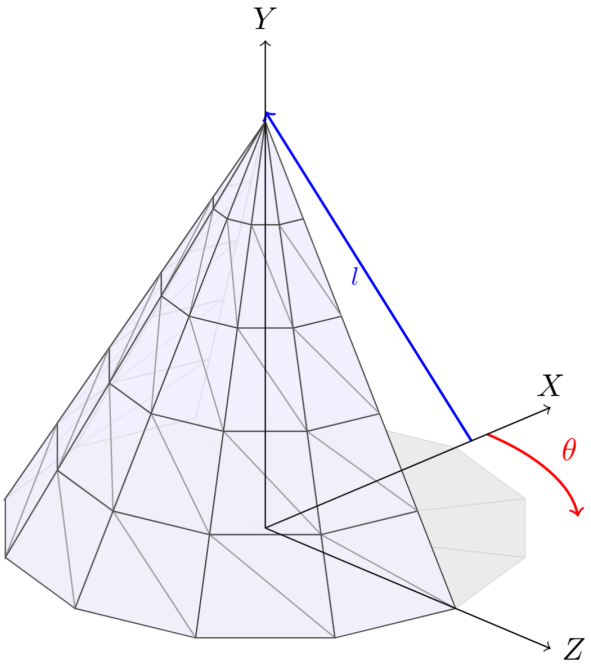


Figura 6: Representação do cone.

$$\begin{cases} p_1 = \left(r_i * \cos(\theta_1), i * \frac{h}{p}, r_i * \sin(\theta_1)\right) \\ p_2 = \left(r_i * \cos(\theta_2), i * \frac{h}{p}, r_i * \sin(\theta_2)\right) \\ p_3 = \left(r_{i+1} * \cos(\theta_1), (i+1) * \frac{h}{p}, r_{i+1} * \sin(\theta_1)\right) \\ p_4 = \left(r_{i+1} * \cos(\theta_2), (i+1) * \frac{h}{p}, r_{i+1} * \sin(\theta_2)\right) \end{cases}$$

Similarmente ao que foi feito para a esfera, no cone fazemos uma pequena otimização no desenho do topo da pirâmide. Quando $i = p - 1$, desenhamos apenas um triângulos ao invés de *quads*.

2.6. Cilindro

Como primitiva adicional simples, decidimos adicionar um cilindro. Este recebe, como parâmetros, o raio da base, seja r, a altura, seja h, o número de fatias (*slices*), seja s e o número de pilhas (*stacks*), seja p.

Tendo desenhado a base do cone, como se vê na Figura 5, criar ambas as bases do cilindro é similar. Usando um ângulo θ, (em que $0^{\text{o}} \leq \theta \leq 360^{\text{o}}$), iteramos pelo número de *slices*, usando $\Delta\theta = \frac{2\pi}{s}$ e fazendo $\theta_1 = i * \Delta\theta$ e $\theta_2 = (i+1) * \Delta\theta$.

A cada passo desta iteração podemos desenhar todos os componentes do cilindro. Desenhamos o triângulo da base:

$$(r * \cos(\theta_1), 0, z * \sin(\theta_1)), (r * \cos(\theta_2), 0, z * \sin(\theta_2)), (0, 0, 0)$$

O triângulo do topo:

$$(r * \cos(\theta_1), h, z * \sin(\theta_1)), (0, h, 0), (r * \cos(\theta_2), h, z * \sin(\theta_2)),$$

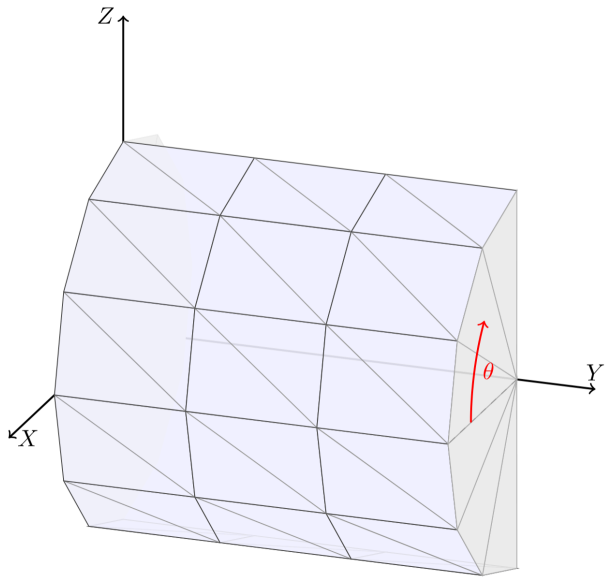


Figura 7: Representação do cilindro.

Para os *quads* da face, dividimos a altura em p *stacks* iguais, de modo a que cada uma tenha uma diferença de alturas de $\Delta h = j * \frac{h}{p}$. Num ciclo interno, mantendo os

mesmos ângulos θ_1 e θ_2, definimos o raio constante r para cada *stack*. Assim, tendo $h_1 = j * \frac{h}{p}$ e $h_2 = (j+1) * \frac{h}{p}$, conseguimos definir os pontos correspondentes da Figura 1 sendo $p_1 = (r * \cos(\theta_2), h_1, r * \sin(\theta_2))$, $p_2 = (r * \cos(\theta_1), h_1, r * \sin(\theta_1))$, $p_3 = (r * \cos(\theta_2), h_2, r * \sin(\theta_2))$ e $p_4 = (r * \cos(\theta_1), h_2, r * \sin(\theta_1))$ [3].

Na figura Figura 7 Mostra-se uma representação desta construção.

2.7. Esfera gerada através de um icosaedro

Por curiosidade desafiamo-nos a tentar utilizar uma abordagem diferente para criar uma aproximação à esfera. Esta ideia veio de observar que o método utilizado na Secção 2.4, criava uma sobrecarga de triângulos pequenos nos polos, enquanto que no centro da esfera havia triângulos maiores, sendo uma aproximação grosseira. Isto deve--se ao facto das *stacks* serem todas do mesmo tamanho. Com um pouco de pesquisa descobrimos que existe um método de aproximação à esfera que parte de um icosaedro.

O icosaedro é um poliedro convexo com 20 faces e 12 vértices (veja-se na Figura 8). É um sólido platónico e desde a Antiguidade era visto como uma aproximação razoável à esfera, visto ter um número significativo de faces e três esferas concêntricas (esfera circunscrita, esfera inscrita e meia-esfera).

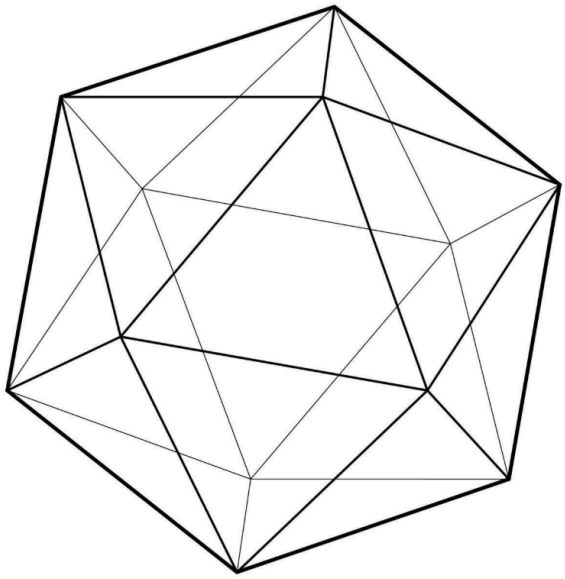


Figura 8: Representação de um icosaedro.

Para o representar, começam-se por definir os 12 vértices iniciais que o constituirão e normaliza-los com base no seu raio, seja r [4]. Apresentam-se abaixo, tendo em atenção que Φ representa a proporção áurea. Em termos práticos, utilizou-se uma aproximação a esta, $\Phi = \frac{1+\sqrt{5}}{2}$. Tenha-se ainda atenção a normalização, feita com auxílio da norma do vetor, $l = \sqrt{x_n^2 + y_n^2 + z_n^2}$, através de $\left(\frac{x*r}{l}, \frac{y*r}{l}, \frac{z*r}{l}\right)$.

$$p_1 = (-1, \Phi, 0), p_2 = (1, \Phi, 0), p_3 = (-1, -\Phi, 0), p_4 = (1, -\Phi, 0),$$

$$p_5 = (0, -1, \Phi), p_6 = (0, 1, \Phi), p_7 = (0, -1, -\Phi), p_8 = (0, 1, -\Phi),$$

$$p_9 = (\Phi, 0, -1), p_{10} = (\Phi, 0, 1), p_{11} = (-\Phi, 0, -1), p_{12} = (-\Phi, 0, 1)$$

Agora, com os vértices definidos, armazena-se a lista extensiva dos seguintes 20 triângulos, que representam as faces:

$$(p_1, p_{12}, p_6), (p_1, p_6, p_2), (p_1, p_2, p_8), (p_1, p_8, p_{11}), (p_1, p_{11}, p_{12}),$$

$$(p_2, p_6, p_{10}), (p_6, p_{12}, p_5), (p_{12}, p_{11}, p_3), (p_{11}, p_8, p_7), (p_8, p_2, p_9),$$

$$(p_4, p_{10}, p_5), (p_4, p_5, p_3), (p_4, p_3, p_7), (p_4, p_7, p_9), (p_4, p_9, p_{10}),$$

$$(p_5, p_{10}, p_6), (p_3, p_5, p_{12}), (p_7, p_3, p_{11}), (p_9, p_7, p_8), (p_{10}, p_9, p_2)$$

Tendo a base dos pontos armazenada, começa-se o processo iterativo, fazendo-o um número de vezes, seja s, passado como argumento, que representa a quantidade de vezes que se efetuará o processo.

Uma iteração do processo consiste em iterar sobre todas as faces do polígono. Para cada face, calculam-se os três pontos médios nas arestas do triângulo, dando origem a quatro novos triângulos, que se vêm na Figura 9. Os vértices v_1, v_2 e v_3 são os originais da face, e os novos triângulos formados serão , (v_1, a, c), (v_2, a, b), (v_3, b, c) e (a, b, c).

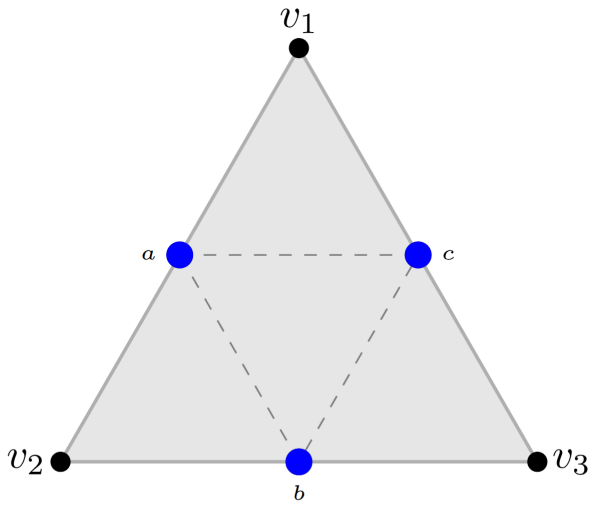


Figura 9: Cálculo dos pontos médios de uma das faces.

O passo essencial surge quando se normalizam, para o raio da esfera, os novos três pontos, a, b e c. Assim, os pontos novos ficam à mesma distância do centro que qualquer outro ponto e todos os triângulos são do mesmo tamanho. Para uma melhor visualização, a Figura 10 mostra os pontos a, b e c após serem normalizados.

Desta maneira algorítmica, conseguimos fazer uma aproximação à esfera que preserva uma uniformidade de geometria, tendo triângulos de igual área e forma similar [5]. Observe-se, na Figura 11, uma aproximação com três iterações deste algoritmo.

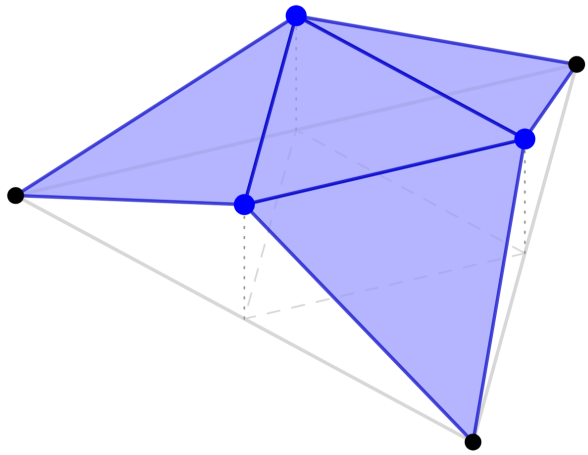


Figura 10: Pontos médios após normalização.

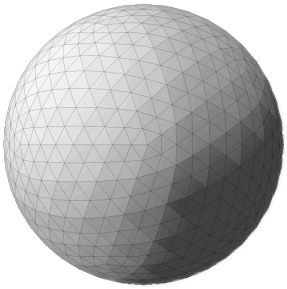


Figura 11: Aproximação à esfera utilizando o método descrito.

2.8. Toro

Por fim fizemos a primitiva do toro. Um toro (*torus*) é um lugar geométrico tridimensional formado pela rotação de uma superfície circular plana de raio r em torno de uma circunferência de raio R.

Este recebe como argumentos a distância da origem até ao centro do tubo, seja R, o raio da circunferência transversal do tubo, seja r, o número de fatias (*slices*), seja s, e o número de camadas, seja p.

Apresenta-se, na Figura 12, a aproximação ao toro que se pretende, nos eixos tridimensionais.

Usar-se-á uma parametrização em função dos ângulos u e v, contidos no domínio $[0, 2\pi]$. O ângulo u representa a rotação em torno do eixo vertical e v a rotação em torno da secção circular do tubo [6]. Mostram-se, estes, na Figura 13, para um melhor entendimento.

Dividimos o espaço da circunferência num tamanho uniforme de *slices* e *stacks*, tendo $\Delta u = \frac{2\pi}{s}$ e $\Delta v = \frac{2\pi}{p}$.

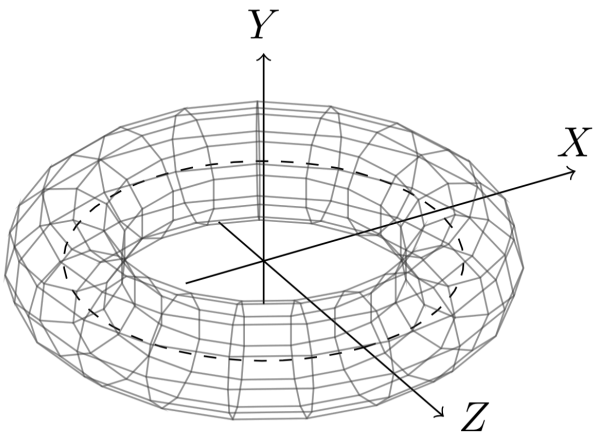


Figura 12: Aproximação desejada de construir do toro.

Itera-se sobre os intervalos de u, usando uma variável auxiliar i, definindo $u_1 = i * \Delta u$ e $u_2 = (i+1) * \Delta u$. Internamente, itera-se sobre os intervalos de v, utilizando a variável auxiliar j, definindo $v_1 = j * \Delta v$ e $v_2 = (j+1) * \Delta v$.

A função paramétrica que usaremos para obter os pontos através dos ângulos define-se como:

$$f(u,v) = ((R + r * \cos(v)) * \cos(u), r * \sin(v), (R + r * \cos(v)) * \sin(u))$$

Podemos, agora, definir os pontos necessários para desenhar os *quads* da construção do toro, sendo esses $p_1 = f(u_1, v_1)$, $p_2 = f(u_2, v_1)$, $p_3 = f(u_1, v_2)$ e $p_4 = f(u_2, v_2)$.

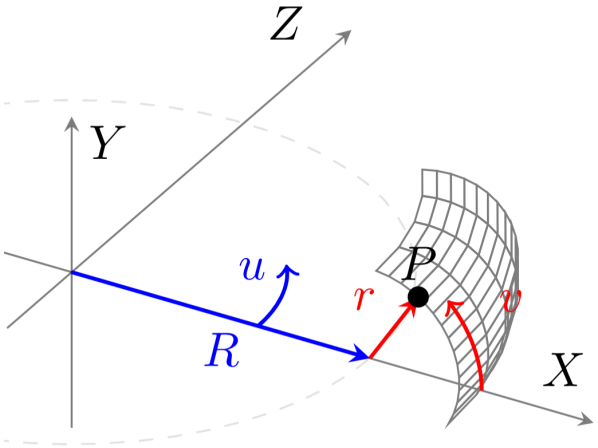


Figura 13: Esquema de representação das variáveis.

Consegue-se, desta maneira, representar o toro.

2.9. Formato dos ficheiros dos modelos

Todos os modelos criados são guardados em ficheiros de extensão *.3d*. Esta formatação foi definida de forma simples e compacta, onde a primeira linha representa o número total de pontos no ficheiro e as subsequentes representações de coordenadas dos pontos com espaçamento. Cada três pontos subsequentes são interpretados como sendo um triângulo.

Veja-se, na Listagem 1, um exemplo simples para a geração de um plano de tamanho e divisão unitárias.

```
12
-0.500000 0.000000 -0.500000
0.500000 0.000000 -0.500000
0.500000 0.000000 0.500000
-0.500000 0.000000 -0.500000
0.500000 0.000000 0.500000
-0.500000 0.000000 0.500000
0.500000 0.000000 -0.500000
-0.500000 0.000000 -0.500000
-0.500000 0.000000 0.500000
0.500000 0.000000 -0.500000
-0.500000 0.000000 0.500000
0.500000 0.000000 0.500000
```

Listagem 1: Arquivo *.3d* resultante da geração de um plano com argumentos 1 e 1.

No exemplo conseguimos ver como a formatação funciona. O valor 12, informa-nos que existem 12 pontos definidos no ficheiro, o que corresponderá a quatro triângulos. Este valor é utilizado pela *engine* para validação.

Esta opção mantém uma estrutura simples, porém eficiente, mantendo uma orientação aos triângulos e uma ordem de pontos contra-relógio, o que garante coerência geométrica.

3. Engine

Com o *generator* explicado, passamos agora ao funcionamento da *engine*.

3.1. Parsing dos dados

Adotamos uma metodologia de total abstração da engine relativamente às primitivas geométricas. Esta conhece apenas triângulos. O programa começa por fazer o parsing do ficheiro XML; para análise sintática destes ficheiros, a biblioteca TinyXML é utilizada, obtendo informações como o comprimento e a largura iniciais da window (tendo em conta que esta pode ser expandida ou reduzida dinamicamente, adaptando a cena através da função `glutReshapeFunction`), bem como toda a informação relevante da câmera — posição, ponto para onde se direciona, vetor up e parâmetros de projeção (FOV, near plane e far plane).

Durante o parsing do ficheiro XML são realizadas verificações de consistência, nomeadamente a validação da existência dos nós obrigatórios, a correta leitura de atributos numéricos e a prevenção de valores inválidos que possam comprometer a configuração da cena.

Adicionalmente, é obtida a lista de ficheiros .3d a serem carregados através da função `loadModel`. Cada ficheiro é lido sequencialmente e a sua geometria é armazenada numa estrutura intermédia de modelo. Para tal, cada ponto é representado por um vértice tridimensional e agregado numa estrutura que identifica o modelo e contém a lista ordenada dos seus vértices:

```
struct Vertex {
    float x, y, z;
};

struct Model {
    string name;
    list<Vertex> vertices;
};
```

Esta abordagem permite desacoplar a leitura dos ficheiros da fase de renderização e facilita a gestão individual de cada modelo. Durante o carregamento são ainda efetuadas verificações como a validação da abertura do ficheiro, o correto formato das linhas lidas e a robustez face a ficheiros vazios ou corrompidos.

3.2. Geração das Figuras

Com todos os dados necessários armazenados, utilizamos a função `renderScene` (registada através da `glutDisplayFunc`) para desenhar os modelos na cena.

Durante o processo de renderização é feita uma iteração sequencial pela lista de vértices de cada Model previamente carregado. Para cada conjunto de três vértices consecutivos é emitido um triângulo. Como os ficheiros .3d foram lidos sequencialmente e os vértices se encontram na ordem correta dentro de cada modelo, é possível gerar corretamente os triângulos individuais.

O desenho é realizado recorrendo a `glBegin(GL_TRIANGLES)`, garantindo que cada trio de vértices forma explicitamente um triângulo na pipeline gráfica. Para otimizar a renderização, é também utilizado back-face culling com orientação CCW (counter--clockwise). Esta abordagem é válida porque o generator garante que os vértices de cada triângulo são armazenados sequencialmente em ordem CCW, permitindo à engine descartar corretamente as faces traseiras e reduzir o número de polígonos processados.

3.3. Interação com a Cena

A *engine* permite ainda a interação dinâmica com a cena, possibilitando ao utilizador explorar os modelos de forma intuitiva. A navegação pode ser realizada tanto através do rato como do teclado.

Com o rato, o utilizador pode rodar a câmara em torno da cena e efetuar *zoom*, permitindo uma inspeção mais detalhada dos objetos. Estas ações são implementadas através do tratamento de eventos de movimento e de botões do rato.

Em alternativa, o teclado disponibiliza controlos adicionais para manipulação da visualização, permitindo ajustar a orientação da câmara e o nível de aproximação. Esta abordagem garante maior flexibilidade e acessibilidade na exploração da cena.

Para maximizar a eficiência do sistema e evitar o processamento redundante da GPU, a aplicação utiliza a função `glutPostRedisplay`. Este mecanismo garante que a cena seja redesenhada apenas quando ocorrem alterações efetivas (como interações do utilizador ou eventos de janela), em vez de manter um ciclo de desenho contínuo que sobrecarregaria os recursos sem necessidade.

4. Resultados obtidos

Segue-se um sumário capítulo para demonstrar os resultados que obtivemos do desenvolvimento desta fase.

4.1. Figuras Geométricas Geradas

Os seguintes modelos foram criados utilizando o *generator* implementado:

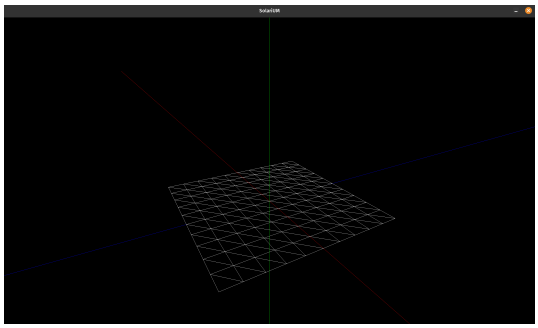


Figura 14: Teste plano.

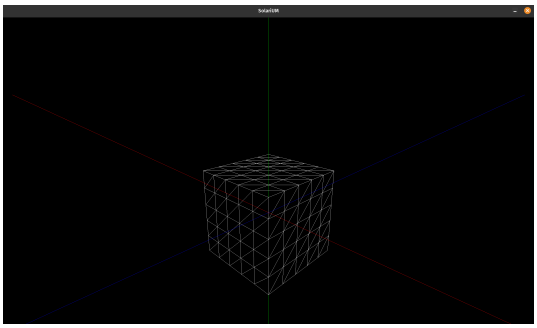


Figura 15: Teste box.

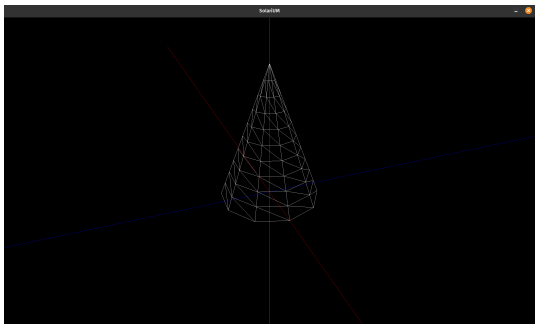


Figura 16: Teste cone.

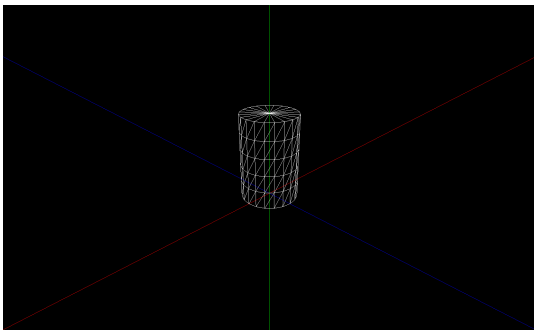


Figura 17: Teste cilindro.

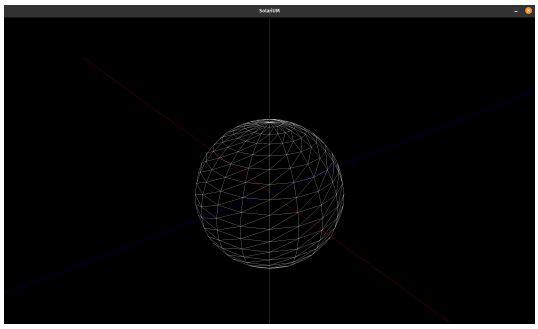


Figura 18: Teste esfera.

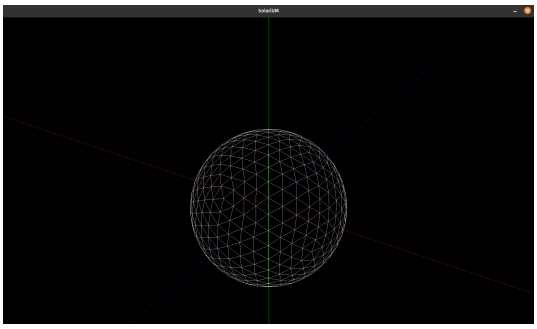


Figura 19: Teste esfera (gerada através de um icosaedro).

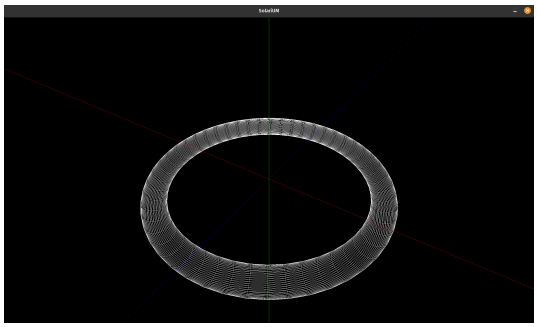


Figura 20: Teste torus.

4.2. Testes Fornecidos

A docência da Unidade Curricular forneceu, juntamente com o enunciado do trabalho algumas cenas a serem testadas no trabalho. A *engine* renderizou-as como esperado:

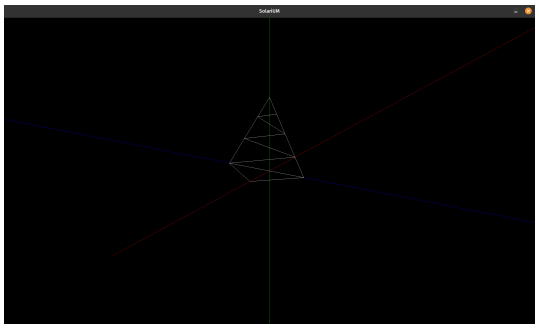


Figura 21: Teste cena 1.

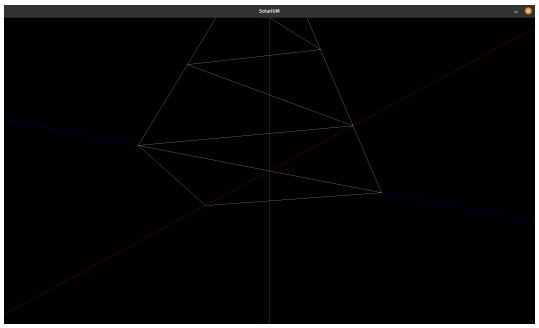


Figura 22: Teste cena 2.

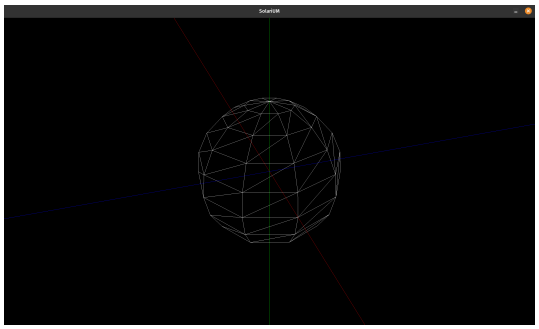


Figura 23: Teste cena 3.

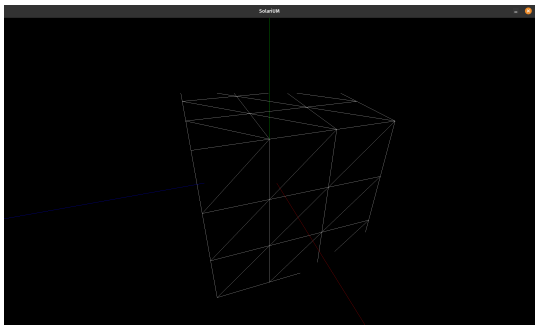


Figura 24: Teste cena 4.

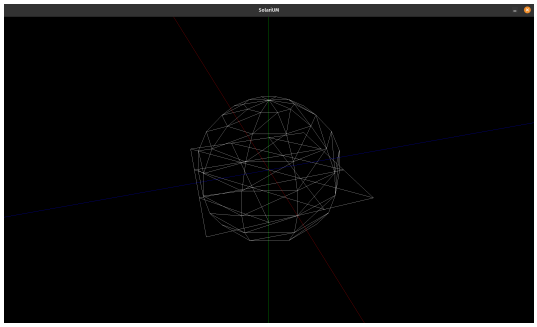


Figura 25: Teste cena 5.

5. Conclusão

A primeira fase do trabalho prático foi realizada com sucesso. Pensando no que foi solicitado pela equipa docente, conseguimos implementar ambos os programas desejados de uma forma pensada e organizada. Fizemos ainda por iniciativa novas primitivas que poderão vir a ser úteis ao decorrer do projeto.

Aplicámos movimentação da câmara, para visualizar de forma mais eficiente os cenários e antecipámos uma otimização para a próxima fase, armazenando os objetos como estruturas individuais de pontos, o que irá facilitar aplicar transformações.

No imediato utilizámos `glBegin(GL_TRIANGLES)`, mas a arquitetura já está preparada para a transição para VBOs (*Vertex Buffer Objects*) nas fases subsequentes, com o intuito de melhorar a performance.

Estamos satisfeitos com o trabalho realizado e os resultados obtidos até então.

Bibliografia

[1] Khronos Group, «OpenGL - The Industry Standard for High Performance Graphics». [Em linha]. Disponível em: https://www.opengl.org/

[2] S. H. Ahn, «Draw Sphere». [Em linha]. Disponível em: https://www.songho.ca/webgl/webgl_sphere.html

[3] S. H. Ahn, «OpenGL Cylinder, Prism & Pipe». [Em linha]. Disponível em: https://www.songho.ca/opengl/gl_cylinder.html

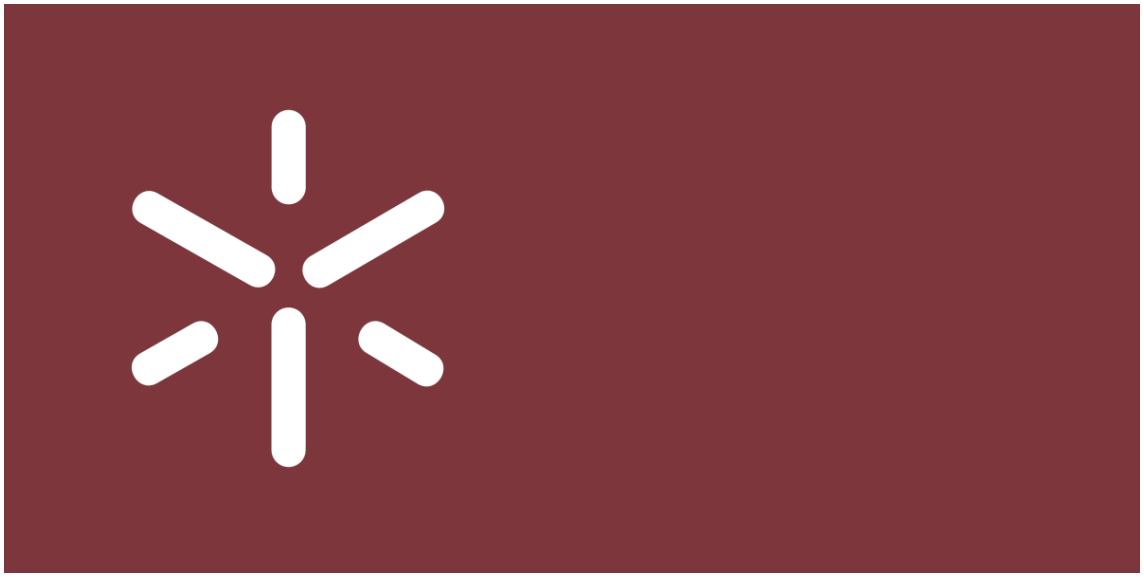
[4] Wikipedia contributors, «Icosahedron». [Em linha]. Disponível em: https://en.wikipedia.org/wiki/Icosahedron

[5] M. Amazonas, «Generating an icosphere with code». [Em linha]. Disponível em: https://blog.lslabs.dev/posts/generating_icosphere_with_code

[6] MathCurve.com, «Torus (Notion géométrique) — MathCurve». [Em linha]. Disponível em: https://mathcurve.com/surfaces.gb/tore/tore.shtml

Lista de Siglas e Acrónimos

API Application Programming Interface
CCW Counter-Clockwise
FOV Field of View
VBO Vertex Buffer Objects
XML Extensible Markup Language

Anexos

Anexo 1: Logo da Universidade do Minho